# Self-Evolving Game Agent with MCTS and Tool-Integration

**Jason Zhu***
UC San Diego
zhuyuezx@ucsd.edu

**Yinuo Zhou***
UC San Diego

**Hongrui (Frank) Zhang***
UC San Diego

**Yaran Yang***
UC San Diego

**Zavier Zhao***
UC San Diego

**Fengfei Yu***
UC San Diego

## Abstract

## 1 Introduction

## 2 Related Work

### 2.1 A Survey of Monte Carlo Tree Search Methods

### 2.2 MC-DML: Monte-Carlo Planning with Dynamic Memory-Guided LLM

### 2.3 Yunjue Agent: A Self-Evolving Agent System for Open-Ended Tasks

## 3 System Architecture

To address the limitations of static heuristic-based planning and the high cost of training deep neural networks for state evaluation, we propose a self-evolving game agent architecture. Our system decouples the environment interaction (gameplay) from the policy improvement mechanism (LLM optimization). The architecture consists of five core components: the RL framework, the tool-evolving LLM agent, the controller, the task proposal loop, and the overall training loop.

### 3.1 RL Framework

The core gameplay engine within our architecture is a pluggable Reinforcement Learning framework. While it is adaptable to algorithms like A* or DQN, we primarily utilize a Monte Carlo Tree Search (MCTS) framework (as illustrated in Figure 2).

Crucially, the framework operates entirely offline relative to the LLM. Step-by-step LLM decision-making during gameplay is inherently slow, costly, and poorly suited for games with implicit semantics derived directly from the board status or game state. In our approach, the RL framework contains zero online LLM calls. Instead, the framework plays games at native compute speeds using programmatic heuristics, passively recording detailed gameplay traces (JSON records) that are stored offline for later optimization (as seen in the Gameplay Records box in Figure 2).

To make the search dynamic and adaptable, we isolate specific decision domains within the MCTS pipeline as optimizable "heuristic slots" (marked as blue boxes in Figure 2). Notably, we found that the **Selection** and **Simulation** phases offer the most significant optimization surfaces:

*Equal contribution.

39th Conference on Neural Information Processing Systems (NeurIPS 2025).

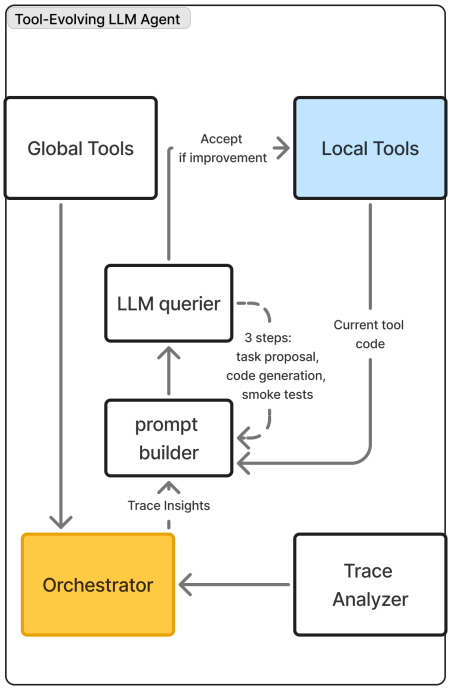


Figure 1: Tool-Evolving LLM Agent pipeline.

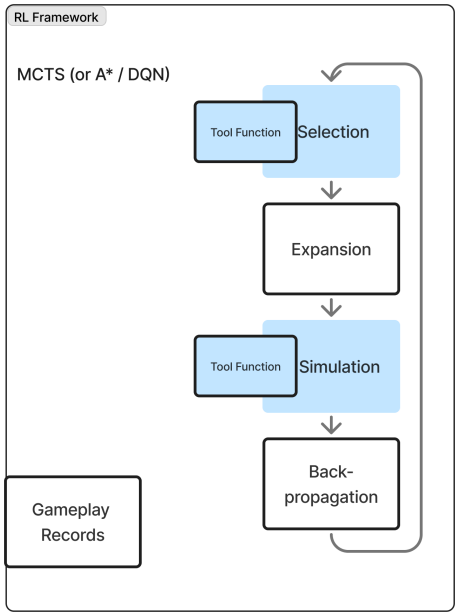


Figure 2: The modular MCTS RL Framework operates offline, using generated Tool Functions for specific phases while passively recording gameplay.

- **Selection:** In traditional MCTS, the best child is selected using a static heuristic such as Upper Confidence bounds applied to Trees (UCT). We encapsulate this selection logic as a programmable tool function, allowing the LLM to write environment-specific rules for exploring the search tree.
- **Simulation:** Standard simulation rollouts often fall back on iteratively selecting uniformly random legal actions until a terminal state is reached. We reinterpret the simulation rollout as a heuristic function in itself. An LLM-designed simulation tool can introduce targeted exploration or score-guided action selection to dramatically improve sample efficiency without requiring deep searches.

These tool functions are encapsulated as standalone Python code snippets that are imported directly by the MCTS engine at runtime. The choice of which function to load for each phase is managed by a central configuration file. Because both the Python snippets and the configuration file are plain text, the LLM can freely create, modify, or hot-swap tools iteratively after reviewing the collected gameplay records.

### 3.2 Tool-Evolving LLM Agent

The LLM serves as an offline strategy improver rather than an online decision-maker. As illustrated in Figure 1, the policy improvement mechanism is driven by several interacting modules designed to translate raw gameplay failures into executable code improvements.

**Trace Analyzer:** The first step in the improvement pipeline is the Trace Analyzer. It acts as an LLM sub-agent that ingests raw game records within a sliding history window (e.g., the last three games by default) and extracts high-level semantics. The Analyzer is tasked with identifying *why* the current strategy succeeded or failed, extracting insights on necessary rule modifications, or determining if a completely novel approach is required.

**Orchestrator and Prompt Builder:** The Orchestrator acts as the central coordinator. It receives insights from the Trace Analyzer, fetches game-specific rules and broad strategies from the Global Tools library, and retrieves the complete source code of the heuristic that produced the failure. The Orchestrator feeds this consolidated context to the Prompt Builder, which constructs the targeted prompts for code generation. By isolating the game-specific logic within Global Tools, the Orchestrator itself remains entirely game-agnostic.

**Multi-Round Query Loop:** To reliably transform abstract reasoning into executable, bug-free Python code, we utilize a multi-round query loop consisting of four distinct stages:

1. **Task Proposal (Analyze):** 1 round. The LLM reviews the Trace Analyzer's insights against the current code to propose a concrete software modification or a new algorithmic approach.
2. **Code Generation (Draft):** 1 round. The LLM writes the initial draft of the proposed Python function.
3. **Refinement (Critique):** 1 round. The drafted code is critiqued for common logic errors, computational efficiency, and proper API usage, resulting in a refined function.
4. **Smoke-Test Fixes:** 1 round. The refined tool is compiled in a sandboxed namespace and run against a dummy environment state (a "smoke test"). If a syntax or runtime exception occurs, the error traceback is fed back to the LLM for a single, immediate repair attempt. If the test passes, this step acts as a no-op.

**Validation and Tool Library Updates:** Following a successful smoke test, the LLM agent waits for the RL framework to execute a set of evaluation "test-runs" using the newly generated code. The Orchestrator compares the test-run performance against the existing baseline (typically a composite score of win rate and sample efficiency).

We only accept the newly generated code if it demonstrates a strict improvement over the current best heuristic. If accepted, the Local Tools library is populated with the new Python snippet (either as a modification of an existing tool or a brand-new creation), and the RL configuration file is updated to point the MCTS engine to this new function. This successful integration and update sequence constitutes one complete optimization loop.

### 3.3 Controller and Game Environment

As depicted in Figure 3, the overall architecture is mediated by the **Controller**, which acts as the core connector between the game itself, the RL framework, and the LLM agent.

**Game Environment and Optimizer:** The underlying game environment is treated as a predefined turn-based simulation. Our framework interfaces with a diverse set of games, including Sokoban, Quoridor, TextWorld, and Zork. The central "Optimizer" controls the RL framework's gameplay loop and orchestrates the interactions between the two primary system modules. Specifically, the Optimizer manages the flow of offline gameplay records from the RL framework to the LLM agent's Trace Analyzer, and conversely handles the dynamic online import of newly generated heuristic tool plugins from the agent back into the search engine.

**Hyperparameters:** Crucially, our architecture explicitly separates the game logic and meta-training configuration from the LLM agent's responsibilities. A dedicated hyperparameters section dictates the game type, level data, training strategy (e.g., number of test evaluation runs), and search algorithm limits (e.g., maximum MCTS rollouts). These are system-level parameters that require human pre-definition and remain strictly outside of the LLM agent's control, ensuring stable and bounded experimentation.

**Game-Agnostic Modularity:** The core competitiveness of this architecture lies in its modularity. Because the game transition logic and the hyperparameter definitions are entirely encapsulated within the top-level Controller, transferring the system to train on a completely different game requires zero changes to the underlying LLM agent or RL codebases. We simply swap the controller-level game endpoint and hyperparameter file while keeping the core modules intact.

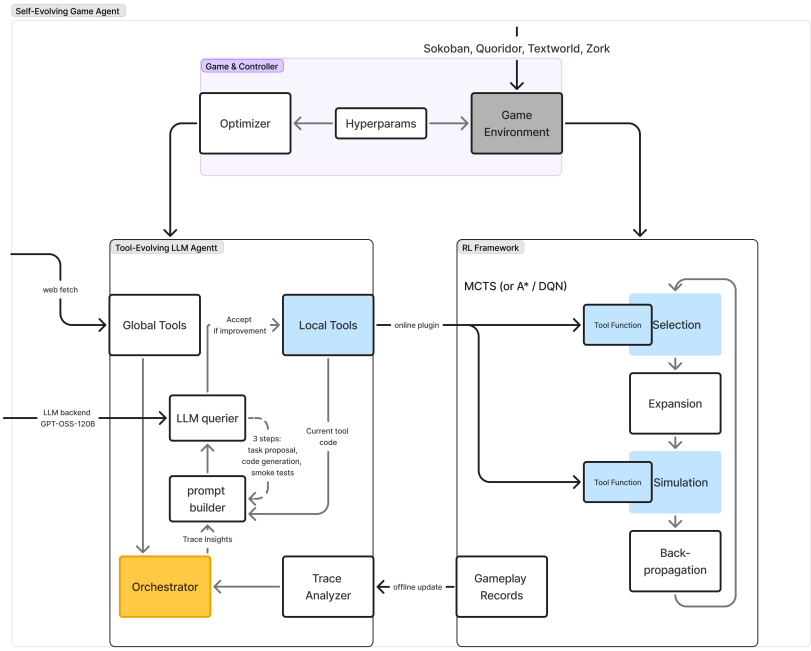


Figure 3: Overall Architecture of the Self-Evolving Game Agent. The top "Game & Controller" module acts as the core connector, governing the interactions between the Tool-Evolving LLM Agent (offline generation) and the RL Framework (online execution/offline logging) via predefined Hyperparameters.

**Framework Flexibility:** Furthermore, the architecture is not strictly tied to MCTS. Any search framework that contains optimizable, programmable components can be plugged into the right-hand "RL Framework" module. For instance, the static heuristic function used in the A* algorithm can perfectly serve as an optimizable "blue box." By targeting the A* distance heuristic for LLM optimization instead of MCTS selection/simulation, the exact same system can evolve performant pathfinding strategies, as we will demonstrate later in Section 4.

### 3.4 Task Proposal Loop

The task proposal loop dictates how the LLM formulates new heuristic ideas, making the initial analysis step the most critical fault point in the system. As defined by our prompt builder module, the analysis phase strictly avoids writing code. Instead, the LLM must first identify "Key Weaknesses" by citing specific gameplay patterns or failure traces, determine the "Root Cause" of these behaviors within the existing heuristic logic, and formally recommend a "Proposed Approach." By design, the LLM is prompted to favor a 70/30 optimization split: approximately 70% of the time, it should recommend an incremental modification to the existing, working code (e.g., adding a deadlock check or tweaking distance weights); the other 30% of the time, it should propose a fundamental paradigm restructure if the current strategy has strictly plateaued. This ensures the system does not get permanently stuck in local-optima parameter tuning.

To extract consistently higher-quality proposals, future iterations could incorporate more advanced prompt engineering techniques directly into this loop. For example, replacing zero-shot analysis with rigorous Few-Shot Chain-of-Thought (CoT) reasoning, where the LLM is provided examples of successful trace-to-insight translations, could yield deeper strategic understanding. Additionally, employing a "debate" framework—where multiple LLM agent Personas independently critique the root cause before agreeing on a proposal—could reduce hallucinated weaknesses and lead to significantly more effective heuristic drafts.

# 4 Experiments

## 4.1 Sokoban (A* Optimization)

### 4.1.1 Sokoban Environment

Sokoban is a classic puzzle video game where the player pushes boxes or crates around in a warehouse, trying to get them to storage locations. The game is notoriously difficult due to its high branching factor and common deadlock states (e.g., pushing a box into a corner). It requires deep planning, making it an ideal testbed for search heuristic optimization.

### 4.1.2 A* Heuristic Optimization

While our core architecture relies on MCTS, we demonstrate its flexibility by plugging an A* search engine into the RL Framework module. Instead of optimizing selection and simulation phases, the LLM agent optimizes the A* cost heuristic function.

Algorithm 1 outlines the core pseudo-code for our A* implementation. The bolded line highlights the exact insertion point where the LLM-generated heuristic code evaluates the board state. Because the framework isolates this logic, the LLM can freely evolve complex strategies without breaking the underlying search mechanics.

**Algorithm 1** Sokoban A* Search with Pluggable Heuristic

```
open_heap ← {initial_state}
closed_set ← ∅
while open_heap ≠ ∅ do
    current ← HeapPop(open_heap)
    if current meets goal conditions then
        return solution
    end if
    if current.hash_key ∈ closed_set then
        continue
    end if
    closed_set ← closed_set ∪ {current.hash_key}
    for each valid move generating new_state do
        new_g ← current.g + 1
        new_h ← heuristic(board, new_state.boxes)        ← Optimized by LLM
        new_state.f ← new_g + new_h
        HeapPush(open_heap, new_state)
    end for
end while
return failure
```

As a baseline, we employ a standard heuristic consisting of a greedy box-to-goal Manhattan distance matching combined with a mobility penalty. The system is run for 10 optimization iterations, with the underlying A* search capped at a maximum of 30,000 search steps per level.

### 4.1.3 Results and Generalization

We evaluated the learning dynamics by comparing two different training curricula: training strictly on the first 20 levels ("Curriculum A") versus randomly sampling 20 levels from a wider pool of 50 levels per iteration ("Curriculum B").

Quantitative results demonstrate that the LLM-driven optimization significantly prunes the A* search space. For Curriculum A, the average node exploration dropped by **74.4%** (from an average of 1,578 nodes in the enhanced baseline to 404 nodes in the optimized version). Curriculum B showed similar prowess, yielding a **64.8%** average reduction across the larger level set.

More impressively, we observed localized cases of extreme improvement where the LLM identified high-impact heuristics (such as advanced deadlock detection and optimal push-distance assignment via DP). For example:

Table 1: Results across configurations on train/test splits. Correct is #correct/#total; Δ is the difference in Correct vs. the baseline on the same split; Ret and Steps are the mean return and mean steps.

| Configuration | Train Correct | Train Δ | Train Ret | Train Steps | Test Correct | Test Δ | Test Ret | Test Steps |
|---|---|---|---|---|---|---|---|---|
| baseline | 62/72 | 0 | 0.861 | 9.9 | 55/90 | 0 | 0.606 | 14.4 |
| + selection | 63/72 | 1 | 0.875 | 12.8 | 75/90 | 20 | 0.833 | 15.8 |
| + expansion | 62/72 | 0 | 0.819 | 10.9 | 55/90 | 0 | 0.478 | 11.3 |
| + simulation | **72/72** | **10** | **1.000** | 41.0 | **90/90** | **35** | **1.000** | 45.1 |
| + backpropagation | 64/72 | 2 | 0.889 | **8.5** | 60/90 | 5 | 0.656 | **10.2** |
| + full | 70/72 | 8 | 0.972 | 14.9 | 89/90 | 34 | 0.989 | 16.2 |

- **Level 2:** Search effort was reduced from 1,223 nodes to just 4 nodes (**99.6%** improvement).
- **Level 6:** Node exploration fell from 9,561 to 33 in the best iteration (**99.6%** improvement).
- **Level 17:** Search count improved from 304 to 17 (**94.4%** improvement).

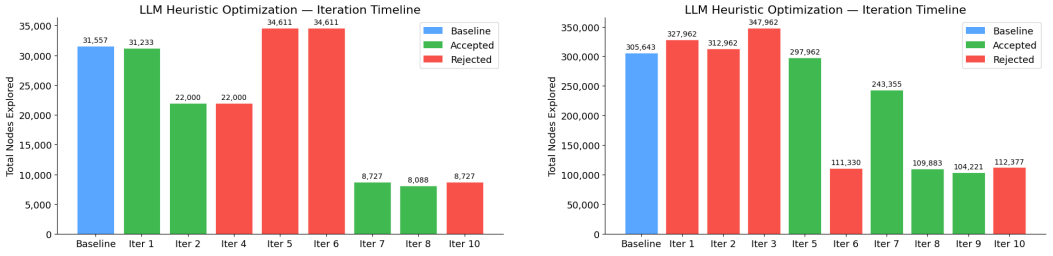


Figure 4: Comparison of the Output 0 metric between straight 20-level training (left) and generalized 50-select-20 training (right).

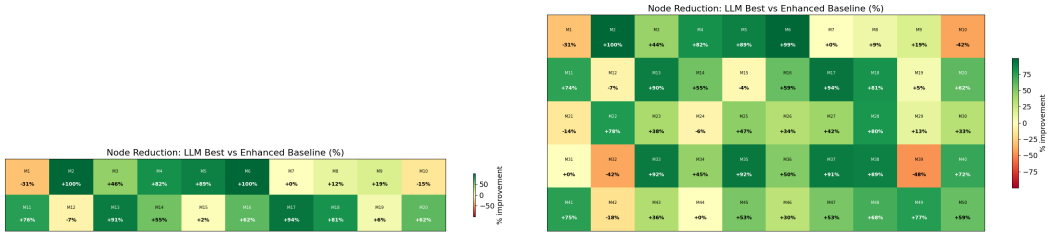


Figure 5: Comparison of the Output 2 metric between straight 20-level training (left) and generalized 50-select-20 training (right).

As shown in Figures 4 and 5, diversifying the training set yields interesting findings regarding the agent's ability to resist overfitting. When the LLM optimizes a heuristic solely on Curriculum A, the generated code strongly exploits level-specific topological oddities. Conversely, the Curriculum B approach forces the prompt builder to provide the LLM with varied failure states across permutations, resulting in a more generalized code heuristic that performs more consistently across larger level distributions.

### 4.2 TextWorld Benchmark

### 4.3 Heuristic Generalizability

### 4.4 Zork

## 5 Limitations

## 6 Future Work

## 7 Conclusion

## References

[1] Browne, C. B., Powley, E., Whitehouse, D., Lucas, S. M., Cowling, P. I., Rohlfshagen, P., Tavener, S., Perez, D., Samothrakis, S., & Colton, S. (2012). A Survey of Monte Carlo Tree Search Methods. *IEEE Transactions on Computational Intelligence and AI in Games*, 4(1), 1–43.

[2] Shi, Z., Fang, M., & Chen, L. (2025). Monte Carlo Planning with Large Language Model for Text-Based Game Agents. *arXiv preprint arXiv:2504.16855*.

[3] Li, H., et al. (2026). Yunjue Agent Tech Report: A Fully Reproducible, Zero-Start In-Situ Self-Evolving Agent System for Open-Ended Tasks. *arXiv preprint arXiv:2601.18226*.

## A Technical Appendix